EEP3020 - Digital Systems Lab

Indian Institute of Technology Jodhpur

Supervisor: Prof. Binod Kumar

Real-Time Human Activity Recognition on STM32F429I-DISC1

Using On-Board Random Forest Inference

Group Members

Name	Roll Number
Sohom Sarkar	B23EE1099
Sambhav Jha	B23EE1092
Rudra Khokhani	B23EE1036
Tula Mrudhul	B23EE1076

April 2026

1. Introduction

Human Activity Recognition (HAR) is the problem of identifying a person's physical activity - walking, climbing stairs, lying down, etc. - from inertial sensor data. It has applications in health monitoring, fall detection, and context-aware computing. In this project we build a complete, self-contained HAR system on the STM32F429I-DISC1 microcontroller. All computation - signal conditioning, feature extraction, and machine-learning inference - runs on the board's ARM Cortex-M4 core at 72 MHz. No host computer is needed during deployment.

The classifier is a Random Forest trained on the UCI HAR dataset and compiled into a static C lookup table. The system classifies the user's activity in real time at roughly one decision per 1.28 seconds and displays the result on the board's built-in 2.4-inch colour LCD.

---

2. System Architecture

```
MPU-6050  ??I2C???  STM32F429I-DISC1
(accel +              ?? Signal conditioning (IIR LPF x 2)
 gyro)                ?? Circular buffer  (128 samples x 9 axes)
                      ?? Feature extraction (284 time+freq features)
                      ?? Feature selection  (100 features)
                      ?? Z-score normalisation
                      ?? Random Forest inference (portable C)
                      ?? Majority vote (3 windows)
                      ?? LCD display  +  UART log
```

Figure 1 - End-to-end pipeline block diagram.

The pipeline is driven by a hardware timer (TIM7) configured to fire an interrupt at exactly 50 Hz. Each interrupt triggers an MPU-6050 I2C read and calls HAR_PushSample(), which advances the pipeline one step. FreeRTOS is used only for task scheduling of the display update loop; all time-critical signal processing happens inside the timer callback context.

---

3. Hardware Platform

Component	Detail
MCU	STM32F429ZIT6, ARM Cortex-M4F, 72 MHz
IMU	InvenSense MPU-6050 (on-board breakout)
Bus	I2C3 - SCL on PA8, SDA on PC9
Display	ILI9341 2.4-inch TFT LCD, 240 x 320 px
UART	USART1 @ 115 200 baud (debug + test log)
IDE	STM32CubeIDE 2.1.1, GCC 14.3 arm-none-eabi

The MPU-6050 is configured with ±2 g accelerometer full-scale range (sensitivity 16 384 LSB/g) and ±250 deg/s gyroscope full-scale range (sensitivity 131 LSB/ deg/s).

Axis remapping. The UCI HAR dataset was collected with the phone flat in a trouser pocket: UCI X ? mediolateral, Y ? gravitational, Z ? anterior-posterior. With our board held in a similar orientation:

```
UCI axis  ?  our MPU axis
  X       ?  AZ  (board depth)
  Y       ?  AY  (board height)
  Z       ?  AX  (board width)
```

This remapping is applied at the point of ingestion: HAR_PushSample(raw.az, raw.ay, raw.ax, raw.gz, raw.gy, raw.gx).

---

4. Signal Processing Pipeline

4.1 Sampling and buffering

Samples arrive at 50 Hz from TIM7's ISR. Each sample is stored in a 9-channel circular ring buffer of depth 128 (= 2.56 s). The nine channels are: noise-filtered accelerometer (3 axes), noise-filtered gyroscope (3 axes), and body acceleration (3 axes, defined below). Every 64 new samples (1.28 s hop) the full 128-sample window is extracted and processed.

4.2 IIR filtering

Two second-order Butterworth biquad filters are applied causally per sample:

- Noise LPF - 18 Hz cutoff. Applied to all six raw IMU axes. Removes sensor noise and aliased high-frequency vibration while preserving the full motion bandwidth (human activities are below ~10 Hz).
- Gravity LPF - 0.3 Hz cutoff. Applied to the three noise-filtered accelerometer axes. The very slow output tracks the quasi-static gravity component along each axis.

Body acceleration is then obtained by subtraction:

```
body_acc = noise_acc ? gravity_acc
```

This removes the ~1 g gravity bias and isolates the inertial dynamics of movement.

4.3 Gain and clipping

Raw body acceleration in the UCI dataset was collected at close to 0 g for stationary subjects and ~0.1-0.2 g RMS for walking. After axis remapping, our board shows slightly lower magnitudes due to placement differences. A gain factor of 6.0 is applied to scale the body acceleration into the range the trained features expect:

```
#define BODY_ACC_GAIN  6.0f
float bax = (nax - gx) * BODY_ACC_GAIN;
```

To prevent brief mechanical taps from inflating the window RMS and triggering a false motion classification, each gained sample is clipped to ±0.5 g before being stored:

```
#define BODY_ACC_CLIP  0.5f
if (bax >  BODY_ACC_CLIP) bax =  BODY_ACC_CLIP;
if (bax < -BODY_ACC_CLIP) bax = -BODY_ACC_CLIP;
```

A sustained walking step produces peaks of ~0.34 g (below clip), contributing fully to the window RMS. A sharp tap lasts 5-10 samples at up to 3 g gained; after clipping to 0.5 g its RMS contribution over 128 samples is ~0.08 g - insufficient to cross the motion threshold.

4.4 Feature extraction

From each 128-sample window, 284 features are computed across all nine channels. The feature set mirrors the UCI HAR feature vector and includes:

- Time domain: mean, standard deviation, median absolute deviation, max, min, signal magnitude area, energy, interquartile range, entropy, correlation (3 axis pairs), auto-regression coefficients (Burg, order 4).
- Frequency domain: via FFT - mean frequency, weighted centroid, energy by band, entropy, maximum component.
- Cross-axis: signal magnitude vector (SMV) of accelerometer and of gyroscope.

4.5 Feature selection and normalisation

A pre-computed mask selects 100 of the 284 features - those identified as most informative during offline training. The selected features are then z-score normalised using pre-stored per-feature mean and standard-deviation parameters (computed on the UCI training set and embedded in flash as float arrays).

4.6 Random Forest inference

The normalised 100-element feature vector is fed to a Random Forest classifier compiled to a C decision-tree structure. The forest was trained on the UCI HAR dataset (10 299 labelled windows from 30 subjects) using scikit-learn. The C code is generated by emlearn (a microcontroller-oriented ML deployment tool). The original model outputs one of six UCI classes:

UCI index	UCI label
0	WALKING
1	WALKING_UPSTAIRS
2	WALKING_DOWNSTAIRS
3	SITTING
4	STANDING
5	LAYING

4.7 Class remapping

SITTING and STANDING are visually and sensorially nearly identical in our deployment context; the UCI dataset

separates them only via subtle postural cues that are sensitive to exact phone placement. WALKING_UPSTAIRS and WALKING_DOWNSTAIRS are likewise difficult to distinguish reliably without a barometer. We therefore remap the six UCI classes to four user-facing classes before the majority vote:

```
static const int RAW_TO_MAPPED[6] = { 1, 2, 2, 0, 0, 3 };
//  UCI:  WALK  UP    DOWN  SIT   STAND LAY
//  Our:  WALK  CLIMB CLIMB STAT  STAT  LAY
```

Our class	Index	Colour
STATIONARY	0	Red
WALKING	1	Cyan
CLIMBING	2	Green
LAYING	3	Purple

4.8 Majority vote

The remapped class label is appended to a circular vote buffer of length 3. The reported activity is the mode (most frequent class) over those 3 windows. This smooths out single-window classification noise and provides a stable display. Confidence is the fraction of the 3-vote buffer that agrees with the majority: 1/3, 2/3, or 3/3 = 33%, 67%, or 100%.

---

5. Software Structure

```
Core/
 ?? Src/
 ?   ?? main.c           - FreeRTOS task, TIM7 ISR, test harness
 ?   ?? har_pipeline.c   - entire signal processing + inference pipeline
 ?   ?? lcd_display.c    - BSP-based LCD update functions
 ?? Inc/
     ?? har_pipeline.h   - public API
     ?? lcd_display.h
```

Key pipeline parameters (compile-time defines in har_pipeline.c):

```
#define SAMPLE_RATE     50      // Hz
#define WINDOW_LEN     128      // samples = 2.56 s
#define HOP_LEN         64      // samples = 1.28 s (50% overlap)
#define VOTE_LEN         3      // majority vote depth
#define N_FEATURES     100      // selected feature count
#define BODY_ACC_GAIN    6.0f   // scale factor
#define BODY_ACC_CLIP    0.5f   // per-sample saturation (g)
```

---

6. LCD Display

```
????????????????????????????????????
?  HAR System  |  STM32F429   ?  ? title bar (cyan)
????????????????????????????????????
?                               ?
?           WALKING             ?  ? class name (class colour, Font24)
?                               ?
????????????????????????????????????
? Confidence:                   ?
? [??????????????????????] 100%  ?  ? bar: green ?70%, orange ?40%, red <40%
????????????????????????????????????
? Acc (g): +0.04 -0.01 +0.99  ?
? Gyr(d/s): +0.3  -1.2  +0.5  ?  ? live sensor readout
????????????????????????????????????
? Samples: 123456  Windows: 45 ?  ? pipeline counters
????????????????????????????????????
```

Figure 2 - LCD layout schematic.

The display is updated at approximately 5 Hz from the FreeRTOS default task. The activity name is only redrawn when the class changes (to avoid flicker); the confidence bar and sensor values are updated every cycle.

---

7. Accuracy Evaluation

7.1 Test procedure

A dedicated firmware test mode (#define HAR_TEST 1 in main.c) was added. On boot, the system automatically cycles through all four activity classes in sequence, holding each for 30 seconds. During each phase the subject performs the labelled activity continuously while the board runs inference. After each phase a 2-second inter-phase gap allows the tester to transition.

Every inference window during the test phase emits one CSV line over UART:

```
phase,window,predicted_class,confidence_pct
```

The test block is bounded by sentinel lines === HAR TEST BEGIN === and === HAR TEST END === to allow automated parsing.

7.2 Settling window discard policy

The first 3 windows of each phase are discarded in the analysis (SETTLE_WIN = 3). This accounts for two sources of initialisation transient:

- Circular buffer fill. At phase start the 128-sample ring buffer still contains samples from the previous phase. The new activity's signal does not fully dominate the window until all 128 slots have been overwritten - approximately 2.56 s.

- Vote buffer warm-up. The 3-window majority vote buffer requires 3 fresh windows (3 x 1.28 s = 3.84 s) to fully stabilise.

Discarding the first 3 windows (3 x 1.28 s = 3.84 s) covers both effects. From window 4 onward, all predictions reflect the current activity exclusively.

7.3 Results

System parameters at test time:

Parameter	Value
Sample rate	50 Hz
Window length	128 samples (2.56 s)
Hop length	64 samples (1.28 s)
Vote buffer depth	3 windows
BODY_ACC_GAIN	6.0x
BODY_ACC_CLIP	0.5 g
Duration per class	30 s
Settling skip	first 3 windows per phase
Valid windows	76 (raw: ~88)

Per-class accuracy:

Class	Correct	Total	Accuracy	Mean Confidence
STATIONARY	18	18	**100.0%**	100%
WALKING	19	19	**100.0%**	100%
CLIMBING	19	19	**100.0%**	100%
LAYING	20	20	**100.0%**	100%
OVERALL	**76**	**76**	**100.0%**	-

Confusion matrix (rows = true class, columns = predicted class):

```
                STATIONARY   WALKING   CLIMBING    LAYING
STATIONARY       18(*)  0         0          0
WALKING              0  19(*)     0          0
CLIMBING             0   0       19(*)       0
LAYING               0   0        0         20(*)
```

All 76 settled windows were classified correctly with 100% vote-buffer confidence. No inter-class confusion was observed.

Timing summary:

Metric	Value
Response period	1.28 s per window (64-sample hop at 50 Hz)

Settling time	~3.84 s (3-window vote buffer fill)
Display update	~5 Hz (FreeRTOS task)

7.4 Discussion

The 100% accuracy result reflects several design decisions working in concert:

- Axis remapping aligns our sensor orientation with the UCI training distribution, avoiding systematic feature mismatch.
- BODY_ACC_GAIN = 6.0 scales the body acceleration into the range the trained features expect, compensating for the different mounting position relative to the original smartphone experiments.
- BODY_ACC_CLIP = 0.5 g prevents brief mechanical taps or table-knock artefacts from transiently inflating the window RMS and triggering a false walking/climbing label.
- Class consolidation (SITTING+STANDING ? STATIONARY, UP+DOWN ? CLIMBING) removes ambiguities that the UCI model cannot reliably resolve in a fixed-orientation wrist/pocket deployment, replacing them with classes that have clear and robust feature signatures.

The four-class problem is inherently simpler than the original six-class one, which partly explains the perfect score. Still, it demonstrates that a full machine-learning inference pipeline - from raw IMU to voted class label - can be realised entirely on a Cortex-M4 at 72 MHz with no floating-point co-processor limitations, running comfortably within the hardware's compute and memory budget.

---

8. Development Journey

The project evolved through several distinct phases:

Phase 1 - Hardware bring-up. I2C communication with the MPU-6050 was verified by reading the WHO_AM_I register. A 50 Hz TIM7 interrupt was configured (prescaler 1439, period 999 at 72 MHz) and confirmed with an oscilloscope.

Phase 2 - Pipeline skeleton. The IIR filter coefficients, circular buffer, and feature extraction code were ported from a reference implementation. Initially the system ran but produced garbage class labels - traced to missing axis remapping (Phase 3).

Phase 3 - Axis remapping. By logging the raw accelerometer outputs while holding the board in various orientations and comparing against the UCI dataset collection protocol, the correct remapping (UCI X = our AZ, etc.) was identified and applied. Class labels became qualitatively sensible immediately.

Phase 4 - Gain calibration. A dedicated gain-tune firmware mode was added that logs (body_rms_g, body_norm) pairs per window. With GAIN = 1.0 all activities hovered near the STATIONARY region. A gain sweep on walking data identified GAIN = 6.0 as placing walking well above the dynamic threshold while keeping stationary safely below it.

Phase 5 - Clipping fix. With GAIN = 6.0, any sharp tap to the board caused the instantaneous body acceleration to exceed 3 g, inflating the 128-sample RMS enough to trigger a false CLIMBING or WALKING classification. Adding per-sample clipping at ±0.5 g resolved this: sustained walking still crosses the threshold; brief taps do not.

Phase 6 - Class consolidation. STANDING was indistinguishable from SITTING regardless of tuning - a known limitation of the UCI model when the sensor is not in the same position as during training. WALKING_UPSTAIRS and WALKING_DOWNSTAIRS were similarly unreliable. The four-class remapping was implemented as a post-inference lookup table with no changes to the trained model.

Phase 7 - LCD display. The BSP ILI9341 driver was integrated. A layered layout was designed: activity name (Font24, class colour), confidence bar (colour-coded green/orange/red), live sensor values, and pipeline counters.

Phase 8 - Accuracy test. A structured 120-second test sequence (4 classes x 30 s) with automatic UART logging was implemented and the results analysed using a Python script, yielding the quantitative results presented in Section 7.

---

9. Conclusion

We have demonstrated a fully self-contained real-time Human Activity Recognition system on the STM32F429I-DISC1. The system:

- Samples a 6-DoF IMU at 50 Hz.
- Applies causal IIR filtering, body-acceleration separation, gain, and clipping.
- Extracts 284 time and frequency domain features per 128-sample window.
- Runs a pre-trained Random Forest classifier entirely in on-chip flash and SRAM.
- Applies a 3-window majority vote and displays the result on a colour LCD.
- Achieves 100% classification accuracy across four activity classes (STATIONARY, WALKING, CLIMBING, LAYING) in a 30 s per class structured accuracy test, with a response latency of 1.28 s per window and a settling time of ~3.84 s.

The project illustrates that edge inference - deploying a trained machine-learning model directly on a resource-constrained microcontroller - is practical at this complexity level, and that careful signal conditioning (axis remapping, gain calibration, clipping) is as important to system performance as the model itself.

Appendix A - Pipeline API Summary

```
// har_pipeline.h
void        HAR_Init(void);
void        HAR_PushSample(int16_t ax, int16_t ay, int16_t az,
                           int16_t gx, int16_t gy, int16_t gz);
int         HAR_GetActivity(void);      // 0-3 or -1
float       HAR_GetConfidence(void);    // 0.0 - 1.0
const char *HAR_GetActivityName(int);
void        HAR_SetTestPhase(int phase); // -1 = disable
```

Appendix B - Key Compile-Time Parameters

Define	Value	Effect

`BODY_ACC_GAIN`	6.0	Body-acc scale factor
`BODY_ACC_CLIP`	0.5 g	Per-sample saturation
`WINDOW_LEN`	128	Window size in samples
`HOP_LEN`	64	Hop size (50% overlap)
`VOTE_LEN`	3	Majority-vote depth
`N_FEATURES`	100	Selected features
`HAR_TEST`	0/1	Enable accuracy-test mode
`TEST_HOLD_MS`	30 000	ms per test phase

Appendix C - Accuracy Test Log Format

```
=== HAR TEST BEGIN ===
# phase,window,predicted,confidence_pct
0,1,0,100
0,2,0,100
...
=== HAR TEST END ===
```

Fields: phase = true class (0-3); window = window index within phase (1-based); predicted = pipeline output (0-3); confidence_pct = vote confidence x 100.

Analysis: python har_test_analysis.py putty.log